UCS420: Cognitive Computing

ASSIGNMENT 7 — NLP-I

Name: Krish Marwaha

Roll No.: 1024170223

Dataset requirement

This assignment must use the tickets and TEST_LINES generated/provided by UCS420_LA9_Starter.ipynb. Run this solution in that notebook after its existing definitions of generate_dataset(), process_ticket(), tickets, and TEST_LINES. Do not modify generate_dataset() or process_ticket(). Since the ticket contents and fingerprint are generated by the starter notebook, ticket-specific output is printed by the code when run with your roll number; fixed sample counts are not substituted here.

Q1. Preprocessing — Run process_ticket() on every ticket. Create a table showing the number of sentences, tokens, punctuation tokens, stopwords, unique tokens, and final tokens for each ticket. Add a total row for count-based columns. For one selected ticket, display actual tokens, punctuation tokens, stopwords, and final tokens.

Answer / Python Code

```
import pandas as pd
import string
from collections import Counter
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords

# Use the tickets and process_ticket() already defined in the starter notebook.
# If your notebook stores tickets in a differently named variable, assign it to tickets.
stop_words = set(stopwords.words("english"))
rows = []
processed = []

for i, ticket in enumerate(tickets, start=1):
    result = process_ticket(ticket)
    processed.append(result)

    # Keep this adapter flexible for a returned dict; adjust keys only if the
    # starter notebook uses different documented key names.
    if isinstance(result, dict):
        tokens = result.get("tokens", word_tokenize(ticket))
        punctuation_tokens = result.get(
            "punctuation_tokens",
            [t for t in tokens if all(ch in string.punctuation for ch in t)]
        )
        stops = result.get(
            "stopwords",
            [t for t in tokens if t.lower() in stop_words]
        )
        final_tokens = result.get("final_tokens", result.get("final", []))
    else:
        # Fallback: calculate visible counts from the raw ticket.
        tokens = word_tokenize(ticket)
        punctuation_tokens = [
            t for t in tokens if all(ch in string.punctuation for ch in t)
        ]
        stops = [t for t in tokens if t.lower() in stop_words]
        final_tokens = [
            t.lower() for t in tokens
            if t not in punctuation_tokens and t.lower() not in stop_words
        ]

    sentences = sent_tokenize(ticket)
    processed[-1] = {
        "raw": ticket,
        "tokens": tokens,
        "punctuation_tokens": punctuation_tokens,
        "stopwords": stops,
        "final_tokens": final_tokens
    }
    rows.append({
        "Ticket": i,
        "Sentences": len(sentences),
```

```
        "Tokens": len(tokens),
        "Punctuation tokens": len(punctuation_tokens),
        "Stopwords": len(stops),
        "Unique tokens": len(set(tokens)),
        "Final tokens": len(final_tokens)
    })

summary = pd.DataFrame(rows)
count_cols = [
    "Sentences", "Tokens", "Punctuation tokens", "Stopwords",
    "Unique tokens", "Final tokens"
]
total_row = {"Ticket": "TOTAL"}
for col in count_cols:
    total_row[col] = int(summary[col].sum())
summary = pd.concat([summary, pd.DataFrame([total_row])], ignore_index=True)
print(summary.to_string(index=False))

# Select the first ticket to show its token lists in detail.
selected = processed[0]
print("\\nSelected ticket:")
print(selected["raw"])
print("Tokens:", selected["tokens"])
print("Punctuation tokens:", selected["punctuation_tokens"])
print("Stopwords:", selected["stopwords"])
print("Final tokens:", selected["final_tokens"])
```

Output

```
A table is printed with one row per generated ticket and a TOTAL row.
The selected ticket's raw text, tokens, punctuation tokens, stopwords,
and final tokens are printed below the table.

Exact rows and counts depend on the ticket dataset generated by the
starter notebook for roll number 1024170223.
```

Q2. Regex extraction and replacement — Using raw tickets and Python re, extract words longer than five letters, numbers including decimals and versions, capitalized words, and words beginning with a vowel. Replace emails, URLs, and phone numbers. Patterns must also work on TEST_LINES.

Answer / Python Code

```
import re

# Join raw ticket texts for dataset-wide demonstrations.
raw_tickets = [str(t) for t in tickets]

patterns = {
    "words_longer_than_5": r"\b[A-Za-z]{6,}\b",
    "numbers_decimals_versions": r"\b\d+(?:\.\d+)+|\b\d+(?:\.\d+)?",
    "capitalized_words": r"\b[A-Z][a-zA-Z]*\b",
    "words_beginning_with_vowel": r"\b[aeiouAEIOU][A-Za-z]*\b"
}

for i, text in enumerate(raw_tickets, start=1):
    print(f"Ticket {i}")
    for label, pattern in patterns.items():
        print(label, ":", re.findall(pattern, text))

email_pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"
url_pattern = r"https?://[^\s<>()]+|www\.[^\s<>()]+"
phone_pattern = r"(?<!\w)(?:\+?\d[\d .()\-]{7,}\d)(?!\w)"

def redact(text):
    text = re.sub(email_pattern, "<EMAIL>", text)
    text = re.sub(url_pattern, "<URL>", text)
    text = re.sub(phone_pattern, "<PHONE>", text)
    return text

print("\\nRegex tests:")
for line in TEST_LINES:
    print("Original:", line)
    for label, pattern in patterns.items():
        print(label, ":", re.findall(pattern, line))
    print("Redacted:", redact(line))
```

Output

```
For every generated ticket and each TEST_LINE, the program prints the
four extraction lists. It then prints each test line before and after
```

```
replacing email addresses with <EMAIL>, URLs with <URL>, and phone
numbers with <PHONE>. Exact extracted lists depend on the supplied text.
```

Q3. Custom tokenization — Implement my_tokenize(text) with one RegexpTokenizer pattern preserving contractions, hyphenated words, decimals, versions, email addresses, URLs, and phone numbers. Run it on TEST_LINES and tickets, compare with word_tokenize(), and explain at least three differences.

Answer / Python Code

```
from nltk.tokenize import RegexpTokenizer, word_tokenize

# Priority in the pattern matters: URL/email/version/decimal/phone first.
token_pattern = (
    r"https?://[^\s<>()]+|www\.[^\s<>()]+|"
    r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}|"
    r"\+?\d[\d .()\-]{7,}\d|"
    r"\d+(?:\.\d+){2,}|\d+\.\d+|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)+|"
    r"[A-Za-z]+|\d+|[^\w\s]"
)
regexp_tokenizer = RegexpTokenizer(token_pattern)

def my_tokenize(text):
    return regexp_tokenizer.tokenize(text)

print("TEST_LINES comparison")
for line in TEST_LINES:
    print("\\nText:", line)
    print("my_tokenize:", my_tokenize(line))
    print("word_tokenize:", word_tokenize(line))

print("\\nTicket comparison")
for i, ticket in enumerate(tickets, start=1):
    print(f"Ticket {i}")
    print("my_tokenize:", my_tokenize(str(ticket)))
    print("word_tokenize:", word_tokenize(str(ticket)))
```

Output

```
The program prints both token lists for each TEST_LINE and ticket.
Inspect these paired lists for dataset-specific examples. Common differences
are explained below; the exact example strings depend on the provided data.
```

Differences to identify:

1. A custom pattern can keep Wi-Fi or state-of-the-art as one token, while a general tokenizer may split at hyphens.
2. The custom pattern can preserve isn't as one token; another tokenizer may split it into isn and 't.
3. The custom pattern can preserve 2.0.1 or 95.5 as one token, while a general tokenizer may split punctuation within the number.

Use the actual paired output above to quote examples that occur in your tickets or TEST_LINES.

Q4. Stemming and lemmatization — On unique final tokens, compare Porter, Lancaster, Regexp stemming, and WordNet lemmatization using POS information from nltk.pos_tag(). Select a Regexp Stemmer suffix based on at least two words in the dataset, justify it, and show only words for which outputs differ.

Answer / Python Code

```
import nltk
from nltk.stem import PorterStemmer, LancasterStemmer, RegexpStemmer, WordNetLemmatizer
from nltk.corpus import wordnet

# Use final tokens created in Q1.
unique_final = sorted({
    str(token).lower()
    for item in processed
    for token in item["final_tokens"]
    if str(token).isalpha()
})

porter = PorterStemmer()
lancaster = LancasterStemmer()
lemmatizer = WordNetLemmatizer()

# Choose a suffix after checking words in your own generated dataset.
# Example: "ing" is suitable only if at least two dataset words end in "ing".
suffix = "ing"
matching_words = [w for w in unique_final if w.endswith(suffix)]
print("Words supporting regexp suffix choice:", matching_words)

regexp_stemmer = RegexpStemmer(suffix + r"$", min=4)

def wordnet_pos(tag):
    if tag.startswith("J"):
        return wordnet.ADJ
    if tag.startswith("V"):
        return wordnet.VERB
    if tag.startswith("N"):
        return wordnet.NOUN
    if tag.startswith("R"):
        return wordnet.ADV
    return wordnet.NOUN

tagged = nltk.pos_tag(unique_final)
comparison = []
for word, tag in tagged:
    values = {
        "Word": word,
        "Porter": porter.stem(word),
        "Lancaster": lancaster.stem(word),
        "Regexp": regexp_stemmer.stem(word),
        "POS Lemma": lemmatizer.lemmatize(word, wordnet_pos(tag))
    }
    if len({values[k] for k in ("Porter", "Lancaster", "Regexp", "POS Lemma")}) > 1:
        comparison.append(values)

print(pd.DataFrame(comparison).to_string(index=False))
print("\\nRegexp suffix justification:")
print("Suffix:", suffix, "| matching words:", matching_words)
print("Use this suffix only if at least two matching dataset words make it reasonable.")
```

Output

```
The code prints words ending in the selected suffix and a table containing
only words whose Porter, Lancaster, Regexp, and POS-aware lemma outputs differ.
The suffix should be verified against the actual dataset before submission.
```

Justification: A RegexpStemmer removes the selected suffix according to a rule; it does not understand a word's meaning. Choose a suffix that occurs in at least two words from your own ticket dataset and mention those words. If fewer than two words end in 'ing', choose a suffix that is genuinely present in at least two dataset words and update the suffix variable.

Q5. Corpus statistics — For S1 (all tokens), S2 (tokens without punctuation), S3 (final tokens), and S4 (S3 after Porter stemming), calculate total tokens, unique tokens, and TTR. Print top 10 words of S1 and S3, plot log(rank) vs log(frequency) for S3, and print top five bigrams of S1 and S3.

Answer / Python Code

```
import math
```

```
import matplotlib.pyplot as plt
from collections import Counter
from nltk.util import bigrams
from nltk.stem import PorterStemmer

# S1: tokens from all tickets using word_tokenize.
S1 = [token for ticket in tickets for token in word_tokenize(str(ticket))]
S2 = [token for token in S1 if not all(ch in string.punctuation for ch in token)]
S3 = [token for item in processed for token in item["final_tokens"]]
porter = PorterStemmer()
S4 = [porter.stem(token) for token in S3]

def corpus_stats(name, tokens):
    total = len(tokens)
    unique = len(set(tokens))
    ttr = unique / total if total else 0
    return {"Corpus": name, "Total tokens": total,
            "Unique tokens": unique, "TTR": round(ttr, 4)}

stats = pd.DataFrame([
    corpus_stats("S1", S1),
    corpus_stats("S2", S2),
    corpus_stats("S3", S3),
    corpus_stats("S4", S4)
])
print(stats.to_string(index=False))

print("\\nTop 10 words of S1:", Counter(S1).most_common(10))
print("Top 10 words of S3:", Counter(S3).most_common(10))
print("Top 5 bigrams of S1:", Counter(bigrams(S1)).most_common(5))
print("Top 5 bigrams of S3:", Counter(bigrams(S3)).most_common(5))

freq = Counter(S3).most_common()
ranks = range(1, len(freq) + 1)
frequencies = [count for word, count in freq]
plt.figure(figsize=(7, 4.5))
plt.plot([math.log(r) for r in ranks],
         [math.log(f) for f in frequencies], marker="o", linestyle="none")
plt.xlabel("log(rank)")
plt.ylabel("log(frequency)")
plt.title("S3: log(rank) vs log(frequency)")
plt.tight_layout()
plt.show()
```

Output

```
The program prints a four-row statistics table, top-10 word lists for S1
and S3, top-five bigram lists for S1 and S3, and a log(rank) vs
log(frequency) scatter plot for S3. Numerical values depend on your generated
tickets and on process_ticket()'s exact preprocessing rules.
```

Q6. Interpretation — (a) Why does TTR change from S1 to S4? (b) Give one example where word_tokenize() and the custom tokenizer differ. (c) Give one example where stemming produces an undesirable result. (d) Give one example where POS-aware lemmatization preserves meaning better than stemming.

Answer

(a) TTR is the number of unique tokens divided by the total number of tokens. Removing punctuation and stopwords changes both counts. Porter stemming merges related word forms into common stems, often reducing the number of unique types in S4; the exact TTR change depends on the dataset.

(b) Use an example shown by the Q3 output. For example, a custom tokenizer may keep state-of-the-art as one token while word_tokenize() splits it at hyphens.

(c) Stemming can produce an undesirable non-word or remove too much of a word. For example, Lancaster stemming is aggressive and may reduce a valid word to a short stem that is difficult to interpret. Replace this with a specific word and output from your comparison table.

(d) POS-aware lemmatization uses grammatical context to find a dictionary base form. For example, the verb 'running' can be lemmatized to 'run' when tagged as a verb, while stemming may produce a stem that is not a proper dictionary word. Confirm the actual result using the printed comparison table.

Run instructions

1. Open the UCS420_LA9_Starter.ipynb notebook from the assignment sheet in Google Colab.
2. Enter roll number 1024170223 and run the starter cells to generate the dataset and TEST_LINES.
3. Run the Q1–Q6 code blocks in order, after the starter definitions. Install/download any NLTK resources requested by the notebook if needed.
4. Use the real printed outputs to complete any examples that explicitly ask for examples from your own dataset. Do not change generate_dataset() or process_ticket().